catanrl v2

experiment log

living document · last updated 17 october 2026 · github.com/mcrco/catanrl-v2 · wandb: catanrl-v2

the goal of this repo is a recipe for the best catan bot out there, for any number of players on any map, with every experiment a config file under `configs/experiments/` and every number below traceable to an eval record under `runs/`. v1 (catanrl) got a two-player bot to about 90% against catanatron's `ValueFunctionPlayer` (F); v2 rewrote the stack around a C++ engine and pufferlib and is past 95% in 2p from scratch. this log is organized by question: what architecture, what the current best recipe is and what it measures, and the list of ideas i'm working through (each with a checkbox).

1 basic arch ablations

previously did some informal arch ablations, and realized that the graph network proposed in canopy is the fastest, most efficient, and most generalizable (can scale to arbitrary maps since you predict board actions per intersection/road/tile). but would still be interested in doing a formal ablation with MLP, LSTM, CNN (like in the old catanrl), and GNN (canopy) eventually.

what the informal ablations (v1, branch `arch-ablation`) actually showed, so i don't have to redo them:

- on a teacher-neutral target (predicting the terminal outcome of held-out games, win/draw/loss cross-entropy in nats, 0.693 is a coin flip) the ranking was graph < MLP < CNN: `graph-s` 0.396 (0.5M params), `mlp-512` 0.464 (2.4M), xdim CNN 0.83 (16.8M). the CNN predicted the winner worse than a coin flip and its value head memorized within a few hundred steps. a graph transformer over the same node and tile tokens (`gtx-s`, 0.397) tied message passing, so what matters is the structured head (per-node settlement/city logits, per-edge road logits, per-tile robber logits), not the exact mixing layer.
- imitation loss is biased toward the teacher's family (search targets carry the teacher's logit shape), so architectures can't be compared on it. that is why the outcome target was used.
- under the full RL recipe (DAgger, then MARL PPO) a 78.5M-parameter CNN replica trailed graph-s by 7–12 points against F at a quarter of the training speed, and graph-m tied graph-s head to head (50.6%) at 2.3× the cost per step. step cost at batch 512: MLP 3 ms, graph-s 26 ms, CNN 120 ms; the graph net is launch-bound (about 200 small kernels per forward), which CUDA graphs fix at inference.

what the formal ablation should look like in v2. the harness already sweeps `model.preset` (`configs/experiments/arch-lineage.yaml` sweeps `graph-s`, `graph-m` and `mlp` over two seeds, but at 400M steps a cell is 6.5 h, so 12 cells is a week of GPU). the v2 row is the engine's padded seat-relative graph row, so the MLP is just a flat network over it (`configs/models/mlp.yaml`, padded dims only) and the CNN has to lay the BASE board out on a grid itself from the tile→node topology the row carries. an LSTM is the odd one out: it changes what the policy sees (a history instead of the latest state), so it is also the first step of the "how markov" idea in Section 3.

- ☑ port v1′s xdim CNN backbone to the v2 row (`src/catanrl/models/cnn.py`, preset `cnn`), BASE-map only like the MLP.
  - done 2026-10-17: v1′s `xdim-default` size (16.8M parameters per network, 15.1M of them in the projection after the convolutions), the tile/node/edge blocks gathered onto v1′s 11 × 21 plane that is laid out once from the engine's BASE map. deviations from v1, all documented in the module: GroupNorm instead of BatchNorm (so rollouts and evals run the same network; `norm: batch` restores v1), every block feature is a channel (47 at 2p) instead of v1′s 20 hand-picked planes, v2′s structured heads. a map with BASE's dims but another tile order (the `hexagon` generator) is refused at launch rather than silently scrambled, which a review caught.
- ☑ an LSTM policy over each seat's row history through pufferlib's `use_rnn` path (preset `lstm`): the trunk embedding feeds an LSTM cell whose state persists across env steps and resets at game ends.
  - done 2026-10-17: the `graph-s` trunk, an LSTM cell over its pooled embedding stepped at each of the seat's own decisions (waiting rows keep their state, a game's first decision starts from zero), the memory injected into both heads; 0.9M parameters. PPO replays the recurrence over segments × horizon from the stored rollout states (CleanRL's PPO-LSTM shape); a 5-epoch run at learning rate 0 reproduces the rollout's logits

exactly (KL 0, clip fraction 0). the native `POLICY` agent keeps one memory stream per seat of a game, so eval records don't depend on `num_envs`; MCTS, BELIEF and the server bridge refuse it for now (a sampled world has no decision history of its own).

- ☑ `configs/experiments/arch-ablation.yaml`: the current best recipe (Section 2) at the fair `private` level, one seed, 100M agent steps per architecture (about 1.6 h each for the graph nets and the mlp, the cnn slower; puffer-graph2 was at 86% vs F by then, and v1′s gaps were 7–12 points, so a 100M-step cell separates them), selected on `select-2p`, reported on `heldout-2p` vs F and head to head against the graph net on the same boards. written and validated 2026-10-17; every cell is `shared: true` except the lstm, which has one trunk by construction.
- ☐ run it. order: `graph-s` first (the other cells' head-to-head rows bind `arch-ablation/graph-s@best`), then `mlp`, `cnn`, `lstm`. queued on the gpu behind the search finals (Section 2).

2 current best recipe

implement engine in c, turn into pufferlib env, run for 1B steps, and get a really good shared-trunk critic + policy that beats `ValueFunctionPlayer` 90% of the time.

put the critic + policy inside a mcts harness, with a simple probabilistic belief-sampling determinization method to get another 5% h2h edge over the raw policy.

2.1 the recipe, concretely

- **engine.** cppanatron, a C++20 port of catanatron kept in lockstep parity with it (2–6 players, explicit maps of any shape). the observation encoder, the action codec, the threaded multi-game env, the hand tracker and the batched MCTS all live in C++, so training, search and deployment can't drift apart. one float32 row per (game, seat), seat-relative, padded to the env's maxima: `[global | players | tiles | nodes | edges | topology | action mask]`.
- **network.** `graph-s`: canopy nexus-v3 style heterogeneous message passing (node↔node, node↔tile, node↔edge), hidden 96, 3 layers, one shared trunk for the policy head and a per-seat value head (one sigmoid per seat = that seat's win probability; the acting row's prediction fills the waiting seats' values, so GAE on each seat's own trajectory costs one network call per game step). 541,540 parameters.
- **training.** pufferlib 3 PPO with two small patches (losses over acting rows only, a value-loss hook), pure self-play (every seat learns), muon at 5e-3 with a cosine to 5e-4, momentum 0.95, $\gamma = 0.999$ per decision, GAE 0.95, 2 update epochs, clip 0.2, entropy 0.005, prioritized 64-step segments, fp32. the batch shape is v1′s graph2 run exactly: 1032 games × 2 rows × 64 steps = 132,096 agent steps per epoch, 16 optimizer steps per epoch, 3786 epochs. `configs/experiments/puffer-graph2.yaml` lists every deviation from v1 and why.
- **search.** `MCTS:ckpt=…:sims=N:dets=K`: a batched N-player PUCT search (per-seat values backed up max^n, exact chance nodes for dice, dev-card draws and steals) over K worlds sampled from the hand tracker's belief at the root (`dets=K`; `oracle=true` searches the true game and is only a reference). `BELIEF:…:worlds=16` is the raw policy averaged over 16 sampled worlds, the fair way to play a network that was trained on more than its seat sees.

2.2 how things are measured

every number is on a held-out suite of boards that training never saw (`validate` checks the training pools against them): `heldout-2p` is 2,500 boards (seed 67) played in both seat orders = 5,000 games; `heldout-4p` is 1,000 boards (seed 68) × 4 rotations = 4,000 games. checkpoints are chosen on a disjoint `select-*` suite every 50 epochs, so the held-out number is never the one that picked the checkpoint. intervals are wilson 95%. in N-player games the number is slot 0′s *score share* (a win counts 1, a truncated game 1/N to every seat), so parity is 1/N. agents that use information a player doesn't have (a `full` policy, an oracle search) are marked; everything else is fair.

2.3 results so far

Table 1: the current best agents against F. v2 trains from scratch (no DAgger) and lands 5 points above v1's warm-started reference in 2p. the 2p `full` policy sees F's development cards; its belief-averaged version doesn't and loses nothing (95.06 vs 95.16), which is the same "hidden information is worth under a point" finding as v1. in 4p the shared-trunk network is trained and played at the `private` level, which is the only fair level past two players.

agent	sees	games	vs F	record
2 players, BASE, 15 vp (`puffer-graph2/full@e3350`, 500M agent steps, 7.95 h)				
raw policy, `full` obs	hidden cards	5,000	**95.16** 94.53–95.72	`ff8528a4`
raw policy, belief-averaged (`BELIEF`, 16 worlds)	fair	5,000	**95.06** 94.42–95.63	`67869e7a`
MCTS 64 sims on the true game	hidden cards	500	94.8 92.5–96.4	`3fdae985`
MCTS 256 sims on the true game	hidden cards	500	95.2 93.0–96.8	`609e0aa7`
4 players, BASE, 10 vp (`ppo-4p/main@e3650`, 1B agent steps = 249M decisions, 9.3 h)				
raw policy vs F, F, F (score share, parity 25%)	fair	4,000	**78.55** 77.25–79.79	`e5b302ee`
two copies vs F, F (team share: 40.45 + 52.75)	fair	4,000	93.2	`b07e5bd9`
raw policy vs R, R, R	fair	4,000	100.0 99.9–100.0	`95570c74`
v1 reference (graph-s, DAgger + MARL PPO, 73.7M steps), for history				
raw policy, belief-averaged	fair	2,000	89.65 88.24–90.91	v1 report
MCTS 256, determinized	fair	1,000	90.5 88.5–92.2	v1 report
MCTS 1024 on the true game	hidden cards	1,000	92.7 90.9–94.2	v1 report

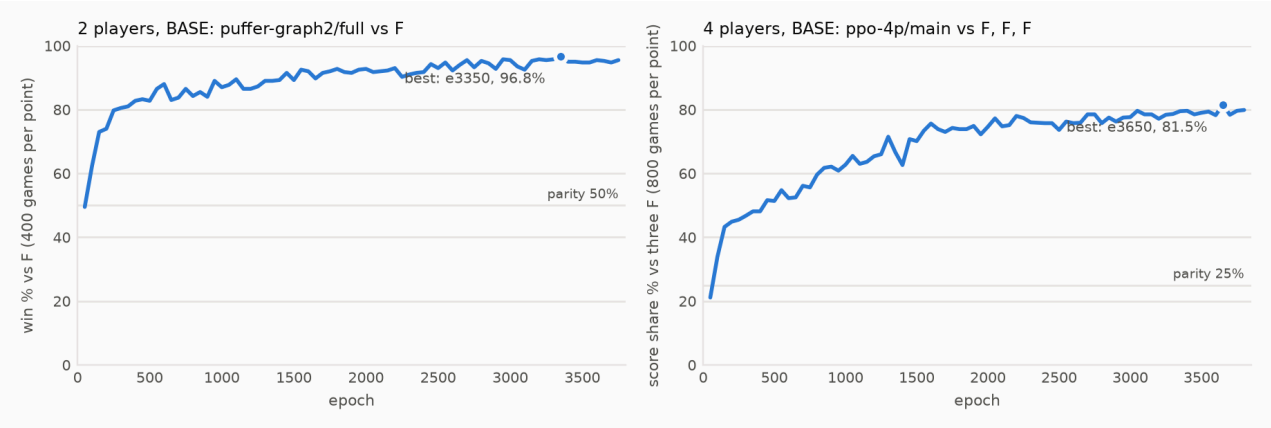


Figure 1: training-time selection curves (argmax policy vs F on the `select-*` suites, every 50 epochs). both runs keep improving through the second half of the cosine, so matched comparisons need the whole budget. full per-epoch logs: puffer-graph2/full and ppo-4p/main on wandb.

the wandb runs were backfilled from `runs/*/metrics.jsonl` and the eval records after the fact (the trainer writes jsonl; wandb mirrors and never decides), so they hold everything the trainer logged: losses, entropy, KL, clip fraction, game length, seat win rates, the critic's calibration (`critic/value_sum_*`: the per-seat win probabilities should sum to about the share of games somebody wins), throughput and the select curve. `ppo-4p`'s critic summed to about 1.2 over its four seats against a realized 0.88, a bias from V-trace targets on waiting rows that `configs/experiments/ppo-4p-critic-fix.yaml` is written to fix (four 16-minute fine-tunes, not run yet). since 2026-10-17 there is a live mirror (`catanrl.wandb_mirror`, on whenever an experiment's `meta.wandb.project` is set; the record on disk stays the source of truth and every wandb failure is a warning) and `tools/wandb_backfill.py` re-syncs any run directory into the same wandb run, so a run launched without the mirror, like `ppo-mixed`, lands there too.

2.4 what is still missing for this section

- ☐ **the 5% search edge in v2, head to head.** v1 measured MCTS against the same network's belief-averaged raw policy on 1,000 games: 51.1% at 256 sims, 55.1% at 1024, 58.1% at 4096 (against F both are near the ceiling, which is why the vs-F rows above don't move). the v2 rows (`MCTS 1024 dets=1` on 500 boards and `MCTS 4096 dets=1` on 250 boards, against `BELIEF` 16 worlds, `heldout-2p`) were started on 2026-10-17 and stopped after ten minutes: sharing the gpu with `ppo-mixed` halved the training's speed and the search had not finished a game, so they resume (per finished game) once that run is done, and will replace this item.
- ☐ **the fair 2p variant.** `puffer-graph2/public` (the `public` level: in 2p every resource movement is seen by both seats, so the opponent's exact hand is a perfect card counter's knowledge, and a `public` policy is fair without

belief averaging) never ran: the gpu queue of 2026-10-14 died after `ppo-4p`. it is the natural bot to serve raw. queued behind `ppo-mixed` (Section 3).
- ☐ the 4p search rows (`MCTS 1024 dets=1` vs three copies of the raw policy on `heldout-4p`), once the critic fix has been checked, since search backs up the critic's values.

3 exploring ideas

the list, with a box per idea; sub-bullets are the thinking so far and what has actually been done. running experiments are dated.

- ☐ **training alphazero** (currently agreement is really high, doesn't seem to yield many gains)
  - the loop exists (`train.algo: alphazero`, `configs/experiments/az-from-ppo.yaml`): self-play through the batched search with gumbel roots, a replay buffer of (row, visit-count policy, per-seat outcome), warm-started from a PPO checkpoint, judged head to head against the network it started from (against F a strong policy is at its ceiling, so vs-F can't see the gain).
  - the one run so far, from a PPO network with 64/16-simulation targets, degraded the policy; no bug found, and shallow search targets can't improve a policy that is already better than a 64-sim search of itself. the next attempt needs ≥ 1024-sim targets, which at 5 s per game-move-batch is expensive: parked until the search edge above is measured, because that number says how much there is to distill at all.
- ☐ **training one model to train for up to `n` players and arbitrary maps** (should be possible by using masked values for numeric features and GNN already handles arbitrary hex boards)
  - the harness already does the masking: the row is padded to the env's maxima with a validity bit per slot, absent seats are all-zero, the global block one-hots the number of seats, and the per-seat value head predicts every present seat. `env.scenarios` draws each game from a weighted list of (player count, rules, map generator).
  - **running since 2026-10-17 02:14** (live on wandb: ppo-mixed/main, re-synced every 20 min): `configs/experiments/ppo-mixed.yaml`, `ppo-4p`'s recipe over six equally weighted scenarios: 2, 3 and 4 players on BASE and on `hexagon` boards (BASE shape, shuffled numbers, generated harbors), same decisions per epoch as `ppo-4p` (3800 epochs, about 10 h). blob-shaped boards of 15–30 tiles (`random-4p`) stay out of training because they would double the row (6316 floats vs 3202) and the memory; the run is evaluated zero-shot on them (`heldout-random-4p`). finals: vs F on `heldout-2p`, `heldout-4p`, `heldout-mixed`, `heldout-random-4p`, and head to head with the specialists (`ppo-4p` on 4p boards, `puffer-graph2`'s belief policy on 2p boards). the question is whether one network matches the specialists at equal decisions; if the blob number is poor, the follow-up trains on blobs with fewer games in flight; 5–6 players (the extension map) comes after, since the engine lacks the special building phase.
- ☐ **adding trading to the engine**
  - the rules side is done: cppanatron implements domestic trades (offer, accept, reject, confirm, cancel) in parity with catanatron. what is missing is everything in front of it: the map-agnostic action space (`GraphActionSpace`) leaves domestic trades out, the env never offers them, and the encoder has no trade-negotiation prompt. so "adding trading" means an action-space extension plus the env, the mask and the trade prompts in the row.
- ☐ **allowing the model to "learn trading"**
  - do not increase action space explosively, either
  - have the model predict the player trade action and a float for each of the 5 types of elements, for both what they give and what they get (11 extra options in the action space), and round to the nearest integer for each of the 10 floats, or
  - have the model predict the player trade action and a float for each of the 18 possible integer choices for each of the 5 types of elements, for both what they give and what they get (181 extra options)
  - i suspect that training while allowing this from scratch will likely not result in the model learning to trade well. instead, we should first train a competent model using the ppo recipe, then take random game states, try applying all every possible legal trades to the current state, and evaluate which trades modify the game state such that the new game state is better for both sides of the trade, according to the critic model
  - plan, in that order: (1) the critic-side analysis first, because it needs no action-space change: sample positions from the trained 2p and 4p networks' self-play, enumerate every 1-for-1 and 2-for-1 trade

between the seat to move and each opponent, apply it to a clone of the position and read both seats' win probabilities from the critic; report how often a trade raises both, by how much, and what the typical trade is. that says whether trading is worth an action space at all under this critic. (2) if it is, the compact head (option one above) for the proposer and a yes/no for the responder, trained from the ppo checkpoint with the trade actions masked in at a low entropy bonus.

- (1) done 2026-10-17: `tools/trade_oracle.py`, results in `report/data/trade-oracle-{ppo-4p-main,puffer-graph2-full}.json`. 3,000 positions each from the network's own self-play (temperature 1, about 3.4 positions per game, 884 / 916 games), every 1:1, 2:1 and 1:2 both hands can pay (about 73 per 4p position), applied through the engine's own offer/accept/confirm path, both seats' win probabilities read from the acting row (the only row shape the critic was trained on; a first version also read the partner's own row and a review showed that to be off-distribution noise). the critic's own noise floor is reported next to every number: its per-seat values don't conserve under a trade (median residual 1.1 pp in 4p, 0.5 pp in 2p), so the 0.5 pp and 1 pp rows are within noise and the 2 pp row is the one to read.
  - 4p (`ppo-4p/main@best`, private): a trade that raises *both* seats' win probability by ≥ 2 pp exists in 6.2% of positions [5.4, 7.2]; by ≥ 1 pp in 25.8% [24.2, 27.5]; by ≥ 5 pp in 0.4%. the best mutual trade gains the mover 1.2 pp and the partner 0.9 pp on average, and costs the two seats not in it 1.1 pp. 92% of mutual trades are 2:1 (the mover gives two; at the private level the mover's row sees the partner's hand only as a size, so this is partly what the critic can see). the mover's selfish best trade gains it 7.8 pp and costs the partner 3.7 pp on average: trades are mostly zero-sum, as expected. most common mutual trade: two wood for an ore.
  - 2p (`puffer-graph2/full@best`, full information): ≥ 2 pp in 0.3% of positions [0.1, 0.5], ≥ 1 pp in 4.3%. two-player catan is nearly zero-sum for this critic, so trading would be an action space that is almost never used.
  - so: worth an action space in 4p (a modest, real pool of 2:1 trades the critic likes for both sides, each costing the other two seats), not in 2p. step (2) waits for the mixed run and your call on the head (option one above is the default).

- ☐ **build the best possible full-information tracker for any number of players** that resolves hidden information (e.g. a robber steal between 2 other players) if either of the other players reveals what happened in the trade via a new action
  - the only hidden information are dev cards and robberies.
  - for example: let us be player A. at the start, we know player B's hand is a wood and a sheep. player C's hand is a brick, an ore, and a wheat. if player B robs player C, and then buys a road, we can resolve that player B now has a sheep, while player C has an ore and a wheat. or if player C robbed player B, and then bought a dev card, we know that player C has a brick, and player B has a wood.
  - another example: let us be player A. we have 6 knights played, player B has 6 knights played, and player C has 1 knight played. then, if player C plays another knight, we know that there are no more knights in anyone else's hand, or in the development card deck, because player C has just played the 14th (and last) knight. we should then never account for any knights in the game. this is important because say if player B has largest army, we know we should stop trying to contend for largest army because we can't get buy any more knight cards. obviously this goes way further than this, since we should be updating our prior on our belief set for the sampling determinization method every time someone plays a dev card.
  - where the current `HandTracker` (C++, `engine/cppanatron/include/cppanatron/tracker.hpp`) stands against these: it keeps, per seat, a known lower bound per resource plus a count of unknown cards, and one global pool of unknown cards derived from the bank, so both robbery examples above should already resolve (when B pays wood + brick the brick comes out of B's unknown card, the pool is left with ore + wheat, and C's two unknown cards can only be those). the knight example is covered on the sampling side (`unseen_development_cards` = deck − my hand − every played card, so after the 14th knight no sampled world holds one) and the row carries every seat's played cards per type plus my own hand, so the network can count too. what it does *not* do: it keeps no prior over *which* card a hidden steal took (a steal from a hand of brick, brick, ore is sampled as brick or ore with equal weight instead of 2:1), it drops evidence from what seats chose not to do, and it is uniform over consistent worlds only up to those two approximations.
  - plan: pin the two examples above as tests, then an exact-posterior tracker that keeps a weighted set of hypotheses over the hidden steals (pruned by every later payment, merged when they reach the same hands, capped with a fall-back to today's pool) and samples worlds from it; measure (a) agreement with

brute-force enumeration on simulated 3p/4p games and (b) `BELIEF` and `MCTS dets=K` vs the current tracker on `heldout-4p`.
  - done 2026-10-17, the tracker and (a): both robbery examples and the 14th-knight example are pinned as engine tests, and the existing pool tracker already resolves all three. the exact mode (`tracker=exact` on `BELIEF` and `MCTS`; `pool` stays the default) keeps weighted hypotheses over hidden steals, branched by the victim's count of each kind, pruned by every later payment, merged when they reach the same hands, capped at 4096 (the most seen in 424 simulated 2–6p games: 748; 2 on average), falling back to the pool past the cap. against brute-force enumeration on 160 positions the exact belief is identical (total variation 0.0); the pool tracker's sampled deal is off by 0.12 (random play) to 0.18 (value play) total variation on average and up to 0.9 in the worst position, so the approximation is real, not just theoretical. sampling a world costs the same (about 12 µs). (b), the `heldout-4p` evals, is queued on the gpu.
- ☐ **train the model to predict the full-information observation from the public-information observation** instead of sampling from belief
  - the natural target already exists: the `full` row of the same position (the env can encode any level), so this is an auxiliary head on the shared trunk predicting the hidden features (opponents' resource and dev-card counts) with a masked regression loss, trained alongside PPO at the `private` level. the test is whether the policy's score moves, not the auxiliary loss. after the mixed run.
- ☐ **define different scopes for how "markov" a model decision is**
  - right now, we only feed the model the latest view of the game state as its input
  - we could instead train a sequential model on the entire game state history, e.g. transformer or lstm
  - this would allow the model to learn things like "if someone didn't play a knight 2 moves ago when one of their high production tiles was blocked, then they probably don't have a knight among their dev cards"
  - this uses many orders of magnitude more compute, but would be only way to get "perfect" bot
  - first step is the LSTM arm of the arch ablation (a recurrent state over the seat's own row history is the cheap end of this scope); a transformer over the whole history is the expensive end and only makes sense if the LSTM beats the markov graph net at `private` level. the lstm class exists since 2026-10-17 (section 1); its 100M-step cell is in the ablation queue.
- ☐ **turn this into an LLM RL env/eval** (e.g. harbor task) and RL a small language model on it, and eval frontier models on how well they play catan
  - `catanllm` (the CS 159 project) already benchmarks LLM players through openrouter against catanatron bots with a few prompting strategies; v2′s game server (protocol 5: committed seeds, signed moves, verifiable transcripts) can seat an LLM client against our bots fairly. an eval is cheap (api cost, no gpu); RL-ing a small model is not on this box.
- ☐ **turn this task of making a strong catan bot an LLM RL env/eval**, and there is a good chance the next frontier models will become amazing at creating better catan bots.

4 log

- **2026-10-14.** `puffer-graph2/full` finished (7.95 h): 95.16% vs F on `heldout-2p`, best checkpoint e3350 by `select-2p`. `ppo-4p/main` launched right after.
- **2026-10-15.** `ppo-4p/main` finished (9.3 h): 78.55% score share vs three F, 100% vs three R. both runs backfilled to wandb. the queue's third job (`puffer-graph2/public`) never started.
- **2026-10-17.** this write-up. launched `puffer-graph2/full`'s missing search finals (MCTS 1024 and 4096 sims, one determinization, vs the belief-averaged policy) and `ppo-mixed` (2–4 players, BASE-shaped boards); the two shared the gpu badly (training at half speed, search at zero finished games in ten minutes), so the finals were stopped and go back in the queue behind the training run: one gpu job at a time from now on. started worktrees for the CNN port, the LSTM policy, the exact tracker, the trade oracle and a wandb mirror; all five were implemented, reviewed and merged into one integration branch the same day (main takes it once the gpu is idle, because the eval provenance hashes the code on disk and a live run refuses to continue under changed code). `ppo-mixed` backfilled to wandb and re-synced every 20 minutes.